PHỤ LỤC - RTL VÀ TESTBENCH ĐẦY ĐỦ

F10 | Lê Chí Hoàng Cường | 24101280. Phụ lục đi kèm bản làm việc, không thay thế logs/CSV/VCD. Mã nguồn có thể tìm theo tên file trong bộ mã chạy lại. Số dòng ở lề chỉ là tham chiếu, không thuộc mã.

A1 - Lõi nguồn, giữ nguyên

f10_handshake_controller.sv

SHA-256: 10cca64461908d5cbe1da1d36be07633c613b92f1c2daeff7fd2ba414cccdfe4

```
 1  module f10_handshake_controller #(
 2      parameter int START_INTERNAL = 0
 3  ) (
 4      input  logic       clk,
 5      input  logic       rst_n,
 6      input  logic       start,
 7      input  logic       ack,
 8      input  logic [7:0] timeout_value,
 9      output logic       req,
10      output logic       done,
11      output logic       timeout,
12      output logic [3:0] state
13  );
14
15      typedef enum logic [3:0] {
16          S_IDLE         = 4'b0000,
17          S_REQ_HIGH     = 4'b0001,
18          S_ACK_HIGH     = 4'b0010,
19          S_WAIT_ACK_LOW = 4'b0011
20      } state_t;
21
22      state_t state_q;
23      logic [7:0] timeout_q;
24      logic [7:0] count_q;
25
26      assign state = state_q;
27      assign req   = (state_q == S_REQ_HIGH) || (state_q == S_ACK_HIGH);
28
29      always_ff @(posedge clk) begin
30          if (!rst_n) begin
31              state_q   <= S_IDLE;
32              timeout_q <= 8'd0;
33              count_q   <= 8'd0;
34              done      <= 1'b0;
35              timeout   <= 1'b0;
36          end else begin
37              done    <= 1'b0;
38              timeout <= 1'b0;
39
40              case (state_q)
41                  S_IDLE: begin
42                      count_q <= 8'd0;
43                      if (start) begin
44                          timeout_q <= timeout_value;
45                          count_q   <= 8'd0;
46                          if (!ack)
47                              state_q <= S_REQ_HIGH;
48                          else
49                              state_q <= S_IDLE;
50                      end
```

```
                end

                S_REQ_HIGH: begin
                    if (ack) begin
                        count_q <= 8'd0;
                        state_q <= S_ACK_HIGH;
                    end else if (timeout_q == 8'd0) begin
                        timeout <= 1'b1;
                        state_q <= S_IDLE;
                    end else if (count_q == timeout_q - 8'd1) begin
                        timeout <= 1'b1;
                        state_q <= S_IDLE;
                    end else begin
                        count_q <= count_q + 8'd1;
                    end
                end

                S_ACK_HIGH: begin
                    count_q <= 8'd0;
                    state_q <= S_WAIT_ACK_LOW;
                end

                S_WAIT_ACK_LOW: begin
                    if (!ack) begin
                        done    <= 1'b1;
                        state_q <= S_IDLE;
                        count_q <= 8'd0;
                    end else if (timeout_q == 8'd0) begin
                        timeout <= 1'b1;
                        state_q <= S_IDLE;
                    end else if (count_q == timeout_q - 8'd1) begin
                        timeout <= 1'b1;
                        state_q <= S_IDLE;
                    end else begin
                        count_q <= count_q + 8'd1;
                    end
                end

                default: begin
                    state_q <= S_IDLE;
                    count_q <= 8'd0;
                    done    <= 1'b0;
                    timeout <= 1'b0;
                end
            endcase
        end
    end

endmodule
```

A2 - Wrapper đã sửa

f10_tiny_tapeout_top.sv

SHA-256: 4d08fe40a7a97fb2b4d9f5236fd8a1ceca01c6e529fb48b6502c569c7b09aa38

```
`default_nettype none

// Working integration assumption: uio_in[1] is START (option 2 in the
// supplied functional_spec). The charter does not explicitly assign START.
// ena suppresses NEW starts only; an accepted handshake must keep progressing.
module f10_tiny_tapeout_top (
    input  wire [7:0] ui_in,
    output wire [7:0] uo_out,
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,
    input  wire       rst_n
);
    wire req, done, timeout;
    wire [3:0] state;

    f10_handshake_controller dut (
        .clk(clk),
        .rst_n(rst_n),
        .start(ena && uio_in[1]),
        .ack(uio_in[0]),
        .timeout_value(ui_in),
        .req(req),
        .done(done),
        .timeout(timeout),
        .state(state)
    );

    assign uo_out  = {1'b0, state, timeout, done, req};
    assign uio_out = 8'b0;
    assign uio_oe  = 8'b0;
    wire _unused = &{uio_in[7:2], 1'b0};
endmodule

`default_nettype wire
```

A3 - Top Tiny Tapeout

tt_um_24101280_f10.sv

SHA-256: 30cfe7059d9e7572ea64f71c638876d24dcdb2a35eafa5c1b32f8415e0b1c5bb

```
1  `default_nettype none
2
3  // Tiny Tapeout top with the student's module name; no extra state or behavior.
4  module tt_um_24101280_f10 (
5      input  wire [7:0] ui_in,
6      output wire [7:0] uo_out,
7      input  wire [7:0] uio_in,
8      output wire [7:0] uio_out,
9      output wire [7:0] uio_oe,
10     input  wire       ena,
11     input  wire       clk,
12     input  wire       rst_n
13 );
14     f10_tiny_tapeout_top impl (
15         .ui_in(ui_in), .uo_out(uo_out), .uio_in(uio_in),
16         .uio_out(uio_out), .uio_oe(uio_oe),
17         .ena(ena), .clk(clk), .rst_n(rst_n)
18     );
19 endmodule
20
21 `default_nettype wire
```

B1 - Testbench lõi tự kiểm

f10_tb_selfcheck.sv

SHA-256: 03bb683e5cd412756abcb5a00318bb7422a4a1d9115e0d9c52aa5cfc907080a4

```
`timescale 1ns/1ps

// Independent, cycle-by-cycle checker for the supplied F10 core contract.
// Drive at falling edges; sample one nanosecond after rising edges, after NBA.
// The model uses a remaining-cycle deadline, never DUT state/internal registers.
module f10_tb_selfcheck;
    logic clk = 0;
    logic rst_n = 0, start = 0, ack = 0;
    logic [7:0] timeout_value = 0;
    wire req, done, timeout;
    wire [3:0] state;

    f10_handshake_controller dut (
        .clk(clk), .rst_n(rst_n), .start(start), .ack(ack),
        .timeout_value(timeout_value), .req(req), .done(done),
        .timeout(timeout), .state(state)
    );
    always #5 clk = ~clk;

    integer model_phase = 0;
    integer model_budget = 1;
    integer remaining = 0;
    logic model_req = 0, model_done = 0, model_timeout = 0;
    integer previous_phase = 0;
    logic sampled_ack = 0;
    logic previous_done = 0, previous_timeout = 0;
    integer busy_age = 0;

    integer cycles = 0, checks = 0, errors = 0;
    integer cases = 0, failed_cases = 0;
    integer case_start_cycle = 0, case_start_checks = 0, case_start_errors = 0;
    integer csv;
    string case_id = "BOOT";
    string case_description = "initial reset";
    time case_start_time;
    integer n, k, phase;
    integer random_resets = 0;
    logic [31:0] rng = 32'hF1002026;

    task automatic check(input logic condition, input string message);
        begin
            checks = checks + 1;
            if (condition !== 1'b1) begin
                errors = errors + 1;
                $display("FAIL case=%s cycle=%0d time=%0t: %s", case_id, cycles,
$time, message);
                $display("  inputs rst_n=%b start=%b ack=%b N=%0d; DUT state=%b
req=%b done=%b timeout=%b; expected phase=%0d req=%b done=%b timeout=%b
remaining=%0d",
                    rst_n, start, ack, timeout_value, state, req, done, timeout,
                    model_phase, model_req, model_done, model_timeout, remaining);
                if (errors >= 20) $fatal(1, "Stopping after 20 failures");
            end
        end
    endtask

    always @(posedge clk) begin
        cycles = cycles + 1;
        previous_phase = model_phase;
        sampled_ack = ack;
```

```
58          model_done = 0;
59          model_timeout = 0;
60
61          if (!rst_n) begin
62              model_phase = 0;
63              model_budget = 1;
64              remaining = 0;
65              busy_age = 0;
66          end else begin
67              case (model_phase)
68                  0: begin
69                      remaining = 0;
70                      if (start && !ack) begin
71                          model_budget = (timeout_value == 0) ? 1 : timeout_value;
72                          remaining = model_budget;
73                          model_phase = 1;
74                      end
75                  end
76                  1: begin
77                      if (ack) begin
78                          model_phase = 2;
79                          remaining = 0;
80                      end else if (remaining <= 1) begin
81                          model_timeout = 1;
82                          model_phase = 0;
83                          remaining = 0;
84                      end else remaining = remaining - 1;
85                  end
86                  2: begin
87                      model_phase = 3;
88                      remaining = model_budget;
89                  end
90                  3: begin
91                      if (!ack) begin
92                          model_done = 1;
93                          model_phase = 0;
94                          remaining = 0;
95                      end else if (remaining <= 1) begin
96                          model_timeout = 1;
97                          model_phase = 0;
98                          remaining = 0;
99                      end else remaining = remaining - 1;
100                 end
101                 default: $fatal(1, "Reference model illegal phase");
102             endcase
103             if (model_phase == 0) busy_age = 0;
104             else busy_age = busy_age + 1;
105         end
106         model_req = (model_phase == 1 || model_phase == 2);
107
108         #1;
109         check((^{state, req, done, timeout}) !== 1'bx, "Outputs must not contain
    X/Z after sampled reset");
110         check(state === model_phase[3:0], "State differs from independent
    reference");
111         check(req === model_req, "REQ differs from independent reference");
112         check(done === model_done, "DONE differs from independent reference");
113         check(timeout === model_timeout, "TIMEOUT differs from independent
    reference");
114         check(state <= 4'd3, "State must be one of the four legal encodings");
115         check(!(done && timeout), "DONE and TIMEOUT must be mutually exclusive");
116         check(!(done && previous_done), "DONE pulse must last only one cycle");
117         check(!(timeout && previous_timeout), "TIMEOUT pulse must last only one
    cycle");
118         check(!(state == 0 && req), "REQ must be low in IDLE");
119         check(!done || (rst_n && previous_phase == 3 && sampled_ack === 1'b0),
120               "DONE requires sampled ACK-low in WAIT_ACK_LOW");
```

```
121         check(busy_age <= 512, "Transaction must terminate within 512 sampled
    cycles");
122         previous_done = done;
123         previous_timeout = timeout;
124     end
125 
126     task automatic cycle(input logic s, input logic a, input integer value, input
    logic reset_n);
127         begin
128             @(negedge clk);
129             start = s;
130             ack = a;
131             timeout_value = value[7:0];
132             rst_n = reset_n;
133             @(posedge clk);
134             #2; // Checker samples at +1 ns, then control returns at +2 ns.
135         end
136     endtask
137 
138     task automatic begin_case(input string id, input string description);
139         begin
140             case_id = id;
141             case_description = description;
142             case_start_cycle = cycles;
143             case_start_checks = checks;
144             case_start_errors = errors;
145             case_start_time = $time;
146         end
147     endtask
148 
149     task automatic end_case;
150         begin
151             cases = cases + 1;
152             if (errors != case_start_errors) failed_cases = failed_cases + 1;
153             $fdisplay(csv, "%s,%s,%0d,%0d,%0d,%0d,%0d,%s", case_id,
    case_description,
154                 case_start_time, $time, cycles - case_start_cycle, checks -
    case_start_checks,
155                 errors - case_start_errors, (errors == case_start_errors) ? "PASS"
    : "FAIL");
156             if (cases <= 10 || (errors != case_start_errors))
157                 $display("CASE %s %s time_ns=[%0d..%0d] cycles=%0d checks=%0d
    errors=%0d",
158                     case_id, (errors == case_start_errors) ? "PASS" : "FAIL",
159                     case_start_time, $time, cycles - case_start_cycle,
160                     checks - case_start_checks, errors - case_start_errors);
161         end
162     endtask
163 
164     task automatic idle_clear;
165         begin
166             cycle(0, 0, 0, 1);
167             check({state, req, done, timeout} === 7'b0, "Idle recovery and result
    pulse clearing");
168         end
169     endtask
170 
171     task automatic complete_handshake(input integer value, input integer delay_up,
    input integer delay_down);
172         integer j;
173         begin
174             cycle(1, 0, value, 1);
175             check(state === 4'd1 && req === 1'b1, "Phase 1 REQ-high");
176             for (j = 0; j < delay_up; j = j + 1) cycle(0, 0, value, 1);
177             cycle(0, 1, value, 1);
178             check(state === 4'd2 && req === 1'b1, "Phase 2 ACK-high accepted while
    REQ stays high");
```

```
179             cycle(0, 1, value, 1);
180             check(state === 4'd3 && req === 1'b0, "Phase 3 REQ-low");
181             for (j = 0; j < delay_down; j = j + 1) cycle(0, 1, value, 1);
182             cycle(0, 0, value, 1);
183             check(done === 1'b1 && timeout === 1'b0, "Phase 4 ACK-low and DONE");
184         end
185     endtask
186 
187     task automatic synchronous_reset_between_edges;
188         logic [6:0] before_reset;
189         begin
190             @(negedge clk);
191             before_reset = {state, req, done, timeout};
192             start = 0;
193             rst_n = 0;
194             #1;
195             check({state, req, done, timeout} === before_reset,
196                   "Synchronous reset must not change outputs before a rising
    edge");
197             @(posedge clk);
198             #2;
199             check({state, req, done, timeout} === 7'b0,
200                   "Synchronous reset clears outputs at the next rising edge");
201             cycle(0, 0, 0, 1);
202         end
203     endtask
204 
205     initial begin
206         $dumpfile("f10_core_selfcheck.vcd");
207         $dumpvars(0, clk, rst_n, start, ack, timeout_value, req, done, timeout,
    state,
208                       model_phase, model_req, model_done, model_timeout, remaining);
209         csv = $fopen("case_results.csv", "w");
210         if (!csv) $fatal(1, "Cannot open case_results.csv");
211         $fdisplay(csv,
    "case_id,description,start_ns,end_ns,cycles,checks,errors,result");
212 
213         begin_case("TC00_RESET", "Reset and idle hold");
214         cycle(0, 0, 0, 0);
215         cycle(0, 0, 0, 1);
216         idle_clear();
217         end_case();
218 
219         begin_case("TC01_NORMAL", "Normal complete four-phase handshake");
220         complete_handshake(8, 0, 0);
221         idle_clear();
222         end_case();
223 
224         begin_case("TC02_DELAYED_SUCCESS", "Delayed ACK-high and ACK-low before
    deadline");
225         complete_handshake(8, 3, 2);
226         idle_clear();
227         end_case();
228 
229         begin_case("TC03_NO_ACK", "No ACK-high timeout after four waiting cycles");
230         cycle(1, 0, 4, 1);
231         repeat (3) begin
232             cycle(0, 0, 4, 1);
233             check(timeout === 1'b0 && req === 1'b1, "No early request-phase
    timeout");
234         end
235         cycle(0, 0, 4, 1);
236         check(timeout === 1'b1 && done === 1'b0, "Request-phase timeout on
    deadline");
237         idle_clear();
238         end_case();
239 
```

```
240         begin_case("TC04_ACK_STUCK", "ACK-high stuck timeout after four waiting
    cycles");
241         cycle(1, 0, 4, 1);
242         cycle(0, 1, 4, 1);
243         cycle(0, 1, 4, 1);
244         repeat (3) begin
245             cycle(0, 1, 4, 1);
246             check(timeout === 1'b0 && state === 4'd3, "No early return-phase
    timeout");
247         end
248         cycle(0, 1, 4, 1);
249         check(timeout === 1'b1 && done === 1'b0, "Return-phase timeout on
    deadline");
250         idle_clear();
251         end_case();
252 
253         begin_case("TC05_BACK_TO_BACK", "Two complete transactions without an idle
    bubble");
254         complete_handshake(2, 0, 0);
255         complete_handshake(9, 3, 2);
256         idle_clear();
257         end_case();
258 
259         begin_case("TC06_STALE_ACK", "Reject start while stale ACK is high and do
    not queue it");
260         repeat (3) begin
261             cycle(1, 1, 3, 1);
262             check(state === 4'd0 && req === 1'b0, "Start with stale ACK is
    rejected");
263         end
264         cycle(0, 0, 3, 1);
265         check(state === 4'd0 && req === 1'b0, "Rejected start is not queued");
266         complete_handshake(3, 0, 0);
267         idle_clear();
268         end_case();
269 
270         begin_case("TC07_BUSY_START", "Ignore start in all three busy states");
271         cycle(1, 0, 5, 1);
272         cycle(1, 0, 1, 1);
273         cycle(1, 1, 255, 1);
274         cycle(1, 1, 0, 1);
275         cycle(1, 1, 1, 1);
276         cycle(1, 0, 255, 1);
277         check(done === 1'b1 && state === 4'd0, "Busy start must not interrupt
    completion");
278         idle_clear();
279         end_case();
280 
281         begin_case("TC08_LATCH_TIMEOUT", "Live timeout input changes do not reload
    either wait deadline");
282         cycle(1, 0, 4, 1);
283         cycle(1, 0, 0, 1);
284         cycle(0, 0, 255, 1);
285         cycle(1, 0, 1, 1);
286         cycle(0, 0, 255, 1);
287         check(timeout === 1'b1, "Request deadline keeps accepted N=4");
288         idle_clear();
289         cycle(1, 0, 4, 1);
290         cycle(1, 1, 1, 1);
291         cycle(1, 1, 0, 1);
292         cycle(0, 1, 255, 1);
293         cycle(1, 1, 1, 1);
294         cycle(0, 1, 0, 1);
295         cycle(1, 1, 255, 1);
296         check(timeout === 1'b1, "Return deadline keeps accepted N=4");
297         idle_clear();
298         end_case();
```

```
299
300         begin_case("TC09_SYNC_RESET", "Reset between edges in IDLE and all three
    busy states");
301         for (phase = 0; phase < 4; phase = phase + 1) begin
302             if (phase > 0) cycle(1, 0, 8, 1);
303             if (phase > 1) cycle(0, 1, 8, 1);
304             if (phase > 2) cycle(0, 1, 8, 1);
305             check(state === phase[3:0], "Reset target phase reached");
306             synchronous_reset_between_edges();
307         end
308         end_case();
309
310         // Every value of the 8-bit configuration domain, both waiting phases,
311         // both deadline outcomes. N=0 means one first waiting-edge decision.
312         for (n = 0; n < 256; n = n + 1) begin
313             begin_case($sformatf("SW_REQ_TIMEOUT_N%03d", n), "Request wait timeout
    at exact deadline");
314             cycle(1, 0, n, 1);
315             for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 0, n, 1);
316             cycle(0, 0, n, 1);
317             check(timeout === 1'b1 && done === 1'b0, "Request sweep exact
    timeout");
318             idle_clear();
319             end_case();
320
321             begin_case($sformatf("SW_LOW_TIMEOUT_N%03d", n), "Return wait timeout
    at exact deadline");
322             cycle(1, 0, n, 1);
323             cycle(0, 1, n, 1);
324             cycle(0, 1, n, 1);
325             for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 1, n, 1);
326             cycle(0, 1, n, 1);
327             check(timeout === 1'b1 && done === 1'b0, "Return sweep exact timeout");
328             idle_clear();
329             end_case();
330
331             begin_case($sformatf("SW_REQ_PRIORITY_N%03d", n), "ACK-high wins over
    timeout on exact deadline");
332             cycle(1, 0, n, 1);
333             for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 0, n, 1);
334             cycle(0, 1, n, 1);
335             check(state === 4'd2 && timeout === 1'b0, "ACK-high priority on request
     deadline");
336             cycle(0, 1, n, 1);
337             cycle(0, 0, n, 1);
338             check(done === 1'b1 && timeout === 1'b0, "Priority transaction
    completes all four phases");
339             idle_clear();
340             end_case();
341
342             begin_case($sformatf("SW_LOW_PRIORITY_N%03d", n), "ACK-low wins over
    timeout on exact deadline");
343             cycle(1, 0, n, 1);
344             cycle(0, 1, n, 1);
345             cycle(0, 1, n, 1);
346             for (k = 1; k < ((n == 0) ? 1 : n); k = k + 1) cycle(0, 1, n, 1);
347             cycle(0, 0, n, 1);
348             check(done === 1'b1 && timeout === 1'b0, "ACK-low priority on return
    deadline");
349             idle_clear();
350             end_case();
351         end
352
353         begin_case("TC10_DETERMINISTIC_STRESS", "4096 synchronous input cycles seed
     F1002026 plus bounded drain");
354         for (k = 0; k < 4096; k = k + 1) begin
355             rng = {rng[30:0], rng[31] ^ rng[21] ^ rng[1] ^ rng[0]};
```

```
356             if (rng[15:8] == 8'h00) random_resets = random_resets + 1;
357             cycle(rng[0], rng[1], rng[23:16], rng[15:8] != 8'h00);
358         end
359         for (k = 0; k < 513 && model_phase != 0; k = k + 1) cycle(0, 0, 0, 1);
360         idle_clear();
361         check(model_phase == 0, "Stress drain leaves no transaction outstanding");
362         end_case();
363 
364         check(cases == 1035, "All directed and sweep cases must execute");
365         check(checks > 0, "A pass requires nonzero checks");
366         check(failed_cases == 0, "No case may fail");
367         $fdisplay(csv, "SUMMARY,all cases,0,%0d,%0d,%0d,%0d,%s", $time, cycles,
    checks, errors,
368             (errors == 0 && failed_cases == 0) ? "PASS" : "FAIL");
369         $fclose(csv);
370         $display("SUMMARY cases=%0d failed_cases=%0d cycles=%0d checks=%0d
    errors=%0d seed=F1002026 stress_resets=%0d",
371             cases, failed_cases, cycles, checks, errors, random_resets);
372         if (errors != 0 || failed_cases != 0) $fatal(1, "F10 SELF-CHECK FAILED");
373         $display("ALL TESTS PASSED: F10 core independent reference and deadline
    sweep");
374         $finish;
375     end
376 
377     initial begin
378         #2000000;
379         $fatal(1, "Global 2 ms simulation watchdog expired");
380     end
381 endmodule
```

B2 - Testbench wrapper tự kiểm

f10_wrapper_tb.sv

SHA-256: 1b02007b98304098460a44a6f6c189f8546c19d4cb3a506eb18d4badc5e3da55

```
`timescale 1ns/1ps
`default_nettype none

// Concrete expected outputs for the complete TT pin interface.
// Stimulus at negedge; checks after following posedge/NBA update.
module f10_wrapper_tb;
    logic clk = 0;
    logic rst_n = 0, ena = 0;
    logic [7:0] ui_in = 0, uio_in = 0;
    wire [7:0] uo_out, uio_out, uio_oe;
    integer checks = 0, errors = 0, cases = 0, failed_cases = 0;
    integer before_errors = 0, before_checks = 0;
    integer csv, k;
    string current_case = "BOOT";
    time begin_time;

    tt_um_24101280_f10 dut (
        .ui_in(ui_in), .uo_out(uo_out), .uio_in(uio_in),
        .uio_out(uio_out), .uio_oe(uio_oe),
        .ena(ena), .clk(clk), .rst_n(rst_n)
    );
    always #5 clk = ~clk;

    task automatic check(input logic condition, input string message);
        checks = checks + 1;
        if (condition !== 1'b1) begin
            errors = errors + 1;
            $display("FAIL case=%s time=%0t %s inputs:rst=%b ena=%b ui=%h uio=%h
out=%h", current_case, $time, message, rst_n, ena, ui_in, uio_in, uo_out);
        end
    endtask

    task automatic outputs(input logic [7:0] expected);
        check((^{uo_out, uio_out, uio_oe}) !== 1'bx, "all output bits known after
reset");
        check(uo_out === expected, $sformatf("full uo_out expected=%h actual=%h",
expected, uo_out));
        check(uio_out === 8'b0, "uio_out tied low");
        check(uio_oe === 8'b0, "all bidirectional pins are inputs");
    endtask

    task automatic step(
        input logic reset_n, enable, start, ack,
        input logic [7:0] deadline,
        input logic [5:0] unused_bits,
        input logic [3:0] expected_state,
        input logic expected_req, expected_done, expected_timeout
    );
        @(negedge clk);
        rst_n = reset_n;
        ena = enable;
        ui_in = deadline;
        uio_in = {unused_bits, start, ack};
        @(posedge clk);
        #1;
        outputs({1'b0, expected_state, expected_timeout, expected_done,
expected_req});
    endtask

    task automatic begin_case(input string id, input string description);
```

```
57          current_case = id;
58          cases = cases + 1;
59          before_errors = errors;
60          before_checks = checks;
61          begin_time = $time;
62          $display("CASE_BEGIN %s %s", id, description);
63      endtask
64
65      task automatic end_case;
66          if (errors != before_errors) failed_cases = failed_cases + 1;
67          $display("CASE_END %s checks=%0d errors=%0d", current_case,
    checks-before_checks, errors-before_errors);
68          $fdisplay(csv, "%s,%0t,%0t,%0d,%0d", current_case, begin_time, $time,
    checks-before_checks, errors-before_errors);
69      endtask
70
71      initial begin
72          $dumpfile("f10_wrapper.vcd");
73          $dumpvars(0, f10_wrapper_tb);
74          csv = $fopen("wrapper_cases.csv", "w");
75          if (!csv) $fatal(1, "cannot open wrapper case CSV");
76          $fdisplay(csv, "case,start_ps,end_ps,checks,errors");
77
78          begin_case("WR01", "synchronous initial reset and complete ports");
79          step(0,1,1,1,8'hff,6'h3f,0,0,0,0);
80          step(1,1,0,0,8'd4,0,0,0,0,0);
81          end_case();
82
83          begin_case("WR02", "ena zero suppresses new start");
84          repeat (3) step(1,0,1,0,8'd4,6'h3f,0,0,0,0);
85          end_case();
86
87          begin_case("WR03", "normal four-phase handshake through mapped pins");
88          step(1,1,1,0,8'd4,0,1,1,0,0);
89          step(1,1,0,0,8'd4,0,1,1,0,0);
90          step(1,1,0,1,8'd4,0,2,1,0,0);
91          step(1,1,0,1,8'd4,0,3,0,0,0);
92          step(1,1,0,0,8'd4,0,0,0,1,0);
93          step(1,1,0,0,8'd4,0,0,0,0,0);
94          end_case();
95
96          begin_case("WR04", "no ACK expires after exactly three wait edges");
97          step(1,1,1,0,8'd3,0,1,1,0,0);
98          step(1,1,0,0,8'd3,0,1,1,0,0);
99          step(1,1,0,0,8'd3,0,1,1,0,0);
100         step(1,1,0,0,8'd3,0,0,0,0,1);
101         step(1,1,0,0,8'd3,0,0,0,0,0);
102         end_case();
103
104         begin_case("WR05", "stale ACK blocks start; fresh start completes
    normally");
105         repeat (2) step(1,1,1,1,8'd4,0,0,0,0,0);
106         step(1,1,0,0,8'd4,0,0,0,0,0);
107         step(1,1,1,0,8'd4,0,1,1,0,0);
108         step(1,1,0,1,8'd4,0,2,1,0,0);
109         step(1,1,0,1,8'd4,0,3,0,0,0);
110         step(1,1,0,0,8'd4,0,0,0,1,0);
111         step(1,1,0,0,8'd4,0,0,0,0,0);
112         end_case();
113
114         begin_case("WR06", "ena falling does not freeze a busy transaction");
115         step(1,1,1,0,8'd4,0,1,1,0,0);
116         step(1,0,1,0,8'd4,0,1,1,0,0);
117         step(1,0,1,1,8'd4,0,2,1,0,0);
118         step(1,0,1,1,8'd4,0,3,0,0,0);
119         step(1,0,1,0,8'd4,0,0,0,1,0);
120         step(1,0,1,0,8'd4,0,0,0,0,0);
```

```
121         end_case();
122
123         begin_case("WR07", "unused uio input bits do not change outputs");
124         step(1,1,0,0,8'd4,6'h3f,0,0,0,0);
125         step(1,1,0,0,8'd4,6'h15,0,0,0,0);
126         step(1,1,0,0,8'd4,6'h2a,0,0,0,0);
127         step(1,1,0,0,8'd4,6'h00,0,0,0,0);
128         end_case();
129
130         begin_case("WR08", "all eight timeout input bits preserved including bit
    seven");
131         step(1,1,1,0,8'h80,0,1,1,0,0);
132         for (k=0; k<127; k=k+1) step(1,1,0,0,8'h80,0,1,1,0,0);
133         step(1,1,0,0,8'h80,0,0,0,0,1);
134         step(1,1,0,0,8'h80,0,0,0,0,0);
135         end_case();
136
137         begin_case("WR09", "zero timeout becomes one wait edge");
138         step(1,1,1,0,8'd0,0,1,1,0,0);
139         step(1,1,0,0,8'd0,0,0,0,0,1);
140         step(1,1,0,0,8'd0,0,0,0,0,0);
141         end_case();
142
143         begin_case("WR10", "stuck high ACK expires in wait-for-low phase");
144         step(1,1,1,0,8'd2,0,1,1,0,0);
145         step(1,1,0,1,8'd2,0,2,1,0,0);
146         step(1,1,0,1,8'd2,0,3,0,0,0);
147         step(1,1,0,1,8'd2,0,3,0,0,0);
148         step(1,1,0,1,8'd2,0,0,0,0,1);
149         step(1,1,0,0,8'd2,0,0,0,0,0);
150         end_case();
151
152         begin_case("WR11", "reset asserted between clocks acts only at next
    posedge");
153         step(1,1,1,0,8'd4,0,1,1,0,0);
154         #1 rst_n = 0;
155         #1 outputs(8'h09); // state=1 and req=1: unchanged before next posedge.
156         @(posedge clk); #1 outputs(8'h00);
157         step(1,1,0,0,8'd4,0,0,0,0,0);
158         end_case();
159
160         $fclose(csv);
161         $display("WRAPPER_SUMMARY cases=%0d failed_cases=%0d checks=%0d
    errors=%0d", cases, failed_cases, checks, errors);
162         if (errors != 0 || failed_cases != 0 || checks == 0) $fatal(1, "wrapper
    test failed");
163         $display("WRAPPER_ALL_TESTS_PASSED");
164         $finish;
165     end
166
167     initial begin
168         #100000;
169         $fatal(1, "wrapper test timeout");
170     end
171 endmodule
172
173 `default_nettype wire
```

B3 - Chẩn đoán deadline và đối tác

f10_contract_bounds_tb.sv

SHA-256: 0929b577ef966f533b77b7a5ad59ed8d9152de2e33d68d4d7cf32a5be2ebfefd

```
`timescale 1ns/1ps

// Supplemental diagnostics only: original 1035-case suite is unchanged.
// Drive at negedge, sample 1 ns after posedge/NBA. Legal ACK stays high
// until REQ has fallen. The last experiment deliberately violates that rule.
module f10_contract_bounds_tb;
    logic clk = 0;
    logic rst_n = 0, start = 0, ack = 0;
    logic [7:0] timeout_value = 0;
    wire req, done, timeout;
    wire [3:0] state;
    integer cycles = 0, checks = 0, errors = 0;
    integer valid_cases = 0, unsupported_observations = 0;
    integer csv;
    bit tracking = 0;
    string current_case = "RESET";
    time req_up_time, ack_up_time, req_down_time, ack_down_time;
    time sampled_edge_time;

    f10_handshake_controller dut (
        .clk(clk), .rst_n(rst_n), .start(start), .ack(ack),
        .timeout_value(timeout_value), .req(req), .done(done),
        .timeout(timeout), .state(state)
    );
    always #5 clk = ~clk;
    always @(posedge clk) cycles = cycles + 1;
    always @(posedge req) if (tracking) req_up_time = $time;
    always @(negedge req) if (tracking) req_down_time = $time;
    always @(posedge ack) if (tracking) ack_up_time = $time;
    always @(negedge ack) if (tracking) ack_down_time = $time;

    task automatic check(input logic condition, input string message);
        checks = checks + 1;
        if (condition !== 1'b1) begin
            errors = errors + 1;
            $display("FAIL case=%s time=%0t cycle=%0d %s state=%b req=%b done=%b
timeout=%b ack=%b",
                current_case, $time, cycles, message, state, req, done, timeout,
ack);
        end
    endtask

    task automatic step(
        input logic s, a, input integer n,
        input logic [3:0] expected_state,
        input logic expected_req, expected_done, expected_timeout
    );
        @(negedge clk);
        start = s;
        ack = a;
        timeout_value = n[7:0];
        @(posedge clk);
        sampled_edge_time = $time;
        #1;
        check({state, req, done, timeout} ===
              {expected_state, expected_req, expected_done, expected_timeout},
              "concrete expected outputs at sampled edge");
        check((^{state, req, done, timeout}) !== 1'bx, "known outputs");
        check(!(done && timeout), "exclusive result events");
    endtask
```

```
59
60      task automatic begin_tracking(input string id);
61          current_case = id;
62          req_up_time = 0;
63          ack_up_time = 0;
64          req_down_time = 0;
65          ack_down_time = 0;
66          tracking = 1;
67      endtask
68
69      task automatic valid_both_deadlines(input integer n);
70          integer w, k, accepted_cycle, latency;
71          time accepted_time, done_time;
72          w = (n == 0) ? 1 : n;
73          begin_tracking($sformatf("VALID_BOTH_DEADLINES_N%03d", n));
74          step(1, 0, n, 1, 1, 0, 0);
75          accepted_cycle = cycles;
76          accepted_time = sampled_edge_time;
77          for (k=1; k<w; k=k+1) step(0, 0, n, 1, 1, 0, 0);
78          // ACK rises on the final permitted request-wait edge.
79          step(0, 1, n, 2, 1, 0, 0);
80          step(0, 1, n, 3, 0, 0, 0);
81          // ACK stays high beyond REQ falling, then falls on the final
82          // permitted return-wait edge. Both phase budgets are consumed.
83          for (k=1; k<w; k=k+1) step(0, 1, n, 3, 0, 0, 0);
84          step(0, 0, n, 0, 0, 1, 0);
85          latency = cycles - accepted_cycle;
86          done_time = sampled_edge_time;
87          check(latency == 2*w+1, "accepted-start to DONE equals two budgets plus
    ACK_HIGH hold");
88          check(req_up_time == accepted_time, "REQ rises on the accepted start
    edge");
89          check(req_up_time < ack_up_time && ack_up_time < req_down_time &&
90                req_down_time < ack_down_time && ack_down_time < done_time,
91                "legal peer: four signal edges strictly ordered before DONE");
92          check((done_time-accepted_time) == latency*10,
93                "cycle latency agrees with raw edge timestamps at 10 ns clock");
94          valid_cases = valid_cases + 1;
95          $display("VALID_PEER_RESULT N=%0d W=%0d accepted_ns=%0d done_ns=%0d
    latency_cycles=%0d edge_ns=[%0d,%0d,%0d,%0d]",
96              n,w,accepted_time,done_time,latency,req_up_time,ack_up_time,req_down_ti
    me,ack_down_time);
97          $fdisplay(csv,"%s,VALID_PEER,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d",
98              current_case,n,w,accepted_time,done_time,latency,
99              req_up_time,ack_up_time,req_down_time,ack_down_time);
100         tracking = 0;
101         step(0, 0, n, 0, 0, 0, 0);
102     endtask
103
104     task automatic observe_unsupported_early_ack_drop;
105         integer accepted_cycle, latency;
106         time accepted_time, done_time;
107         begin_tracking("EARLY_ACK_DROP");
108         step(1, 0, 8, 1, 1, 0, 0);
109         accepted_cycle = cycles;
110         accepted_time = sampled_edge_time;
111         step(0, 1, 8, 2, 1, 0, 0);
112         check(req === 1'b1 && ack === 1'b1, "about to violate peer hold rule during
     ACK_HIGH");
113         // Deliberate unsupported peer behavior: ACK falls at the next
114         // negedge while REQ is still high, before core lowers REQ.
115         step(0, 0, 8, 3, 0, 0, 0);
116         check(ack_down_time < req_down_time, "experiment actually violated ACK hold
     ordering");
117         step(0, 0, 8, 0, 0, 1, 0);
118         latency = cycles - accepted_cycle;
119         done_time = sampled_edge_time;
```

```
120         check(latency == 3, "observed core completes despite unsupported early ACK
    drop");
121         unsupported_observations = unsupported_observations + 1;
122         $display("OBSERVED_UNSUPPORTED_PEER N=8 accepted_ns=%0d done_ns=%0d
    latency_cycles=%0d edge_ns=[%0d,%0d,%0d,%0d] ACK_fell_before_REQ: not a
    valid-protocol PASS",
123             accepted_time,done_time,latency,req_up_time,ack_up_time,req_down_time,a
    ck_down_time);
124
    $fdisplay(csv,"%s,OBSERVED_UNSUPPORTED_PEER,8,8,%0d,%0d,%0d,%0d,%0d,%0d,%0d",
125             current_case,accepted_time,done_time,latency,
126             req_up_time,ack_up_time,req_down_time,ack_down_time);
127         tracking = 0;
128         step(0, 0, 8, 0, 0, 0, 0);
129     endtask
130
131     initial begin
132         $dumpfile("f10_contract_bounds.vcd");
133         $dumpvars(0, f10_contract_bounds_tb);
134         csv = $fopen("diagnostic_cases.csv", "w");
135         if (!csv) $fatal(1,"cannot open diagnostic CSV");
136         $fdisplay(csv,"case_id,classification,N,W,accepted_ns,result_ns,latency_cyc
    les,req_up_ns,ack_up_ns,req_down_ns,ack_down_ns");
137         step(0, 0, 0, 0, 0, 0, 0);
138         rst_n = 1;
139         step(0, 0, 0, 0, 0, 0, 0);
140         valid_both_deadlines(0);
141         valid_both_deadlines(1);
142         valid_both_deadlines(255);
143         observe_unsupported_early_ack_drop();
144         check(valid_cases == 3 && unsupported_observations == 1, "all four
    diagnostic experiments executed");
145         check(checks > 0, "diagnostic checker executed");
146         $fclose(csv);
147         $display("DIAGNOSTIC_SUMMARY valid_cases=%0d unsupported_observations=%0d
    checks=%0d errors=%0d",
148             valid_cases,unsupported_observations,checks,errors);
149         if (errors != 0) $fatal(1,"diagnostic expectation mismatch");
150         $display("DIAGNOSTIC_COMPLETED: valid-peer bounds measured; unsupported
    peer observation is a documented limit");
151         $finish;
152     end
153
154     initial begin
155         #100000;
156         $fatal(1,"100 us diagnostic watchdog expired");
157     end
158 endmodule
```